

Air University



Cryptography (Lab)

Year 2025

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: February 12, 2026

Lab Task # 01**Ceaser Cipher****1. Code**

```
#include <iostream>
#include <string>
#include <cctype> // For isalpha and toupper/tolower
#include <vector> // For storing brute force results
#include <utility> // For pair

using namespace std;

class CaesarCipher
{
private:
    int shift;

public:
    // Constructor to set the shift value
    CaesarCipher(int s) : shift(s % 26) {} // Normalize shift to 0-25

    // Function to encrypt plaintext
    string encrypt(const string &text)
    {
        string result = "";
        for (char c : text)
        {
            if (isalpha(c))
            {
                // Convert to numeric value (0-25)
                char base = isupper(c) ? 'A' : 'a';
                int numeric = c - base;
                // Apply encryption formula: (numeric + shift) % 26
                int encryptedNumeric = (numeric + shift) % 26;
                // Convert back to character
                result += encryptedNumeric + base;
            }
            else
            {
                result += c; // Non-alphabetic unchanged
            }
        }
        return result;
    }

    // Function to decrypt ciphertext
    string decrypt(const string &text)
    {
        string result = "";
```

```
for (char c : text)
{
    if (isalpha(c))
    {
        // Convert to numeric value (0-25)
        char base = isupper(c) ? 'A' : 'a';
        int numeric = c - base;
        // Apply decryption formula: (numeric - shift + 26) % 26
        int decryptedNumeric = (numeric - shift + 26) % 26;
        // Convert back to character
        result += decryptedNumeric + base;
    }
    else
    {
        result += c; // Non-alphabetic unchanged
    }
}
return result;
}

// Function to perform brute force attack on ciphertext
// Returns a vector of pairs: (key, decrypted_text)
vector<pair<int, string>> bruteForceAttack(const string &ciphertext)
{
    vector<pair<int, string>> results;
    // Try all possible keys from 1 to 25
    for (int s = 1; s <= 25; ++s)
    {
        CaesarCipher tempCipher(s);
        // Decrypt using each key
        string decrypted = tempCipher.decrypt(ciphertext);
        results.push_back({s, decrypted});
    }
    return results;
}

// Method to change shift
void setShift(int s)
{
    shift = s % 26;
}

// Method to get current shift
int getShift() const
{
    return shift;
}
};

int main()
{
    int choice;
```

```
do
{
    cout << "\nMenu:" << endl;
    cout << "1. Encrypt" << endl;
    cout << "2. Decrypt" << endl;
    cout << "3. Brute Force Attack" << endl;
    cout << "4. Exit" << endl;
    cout << "Choose an option (1-4): ";
    cin >> choice;
    cin.ignore(); // Clear newline from input buffer

    if (choice == 1)
    {
        // Encryption
        string plaintext;
        int K;
        cout << "Enter plaintext: ";
        getline(cin, plaintext);
        cout << "Enter key K (1-25): ";
        cin >> K;
        cin.ignore(); // Clear newline
        CaesarCipher cipher(K);
        string encrypted = cipher.encrypt(plaintext);
        cout << "Ciphertext: " << encrypted << endl;
    }
    else if (choice == 2)
    {
        // Decryption
        string ciphertext;
        int K;
        cout << "Enter ciphertext: ";
        getline(cin, ciphertext);
        cout << "Enter key K (1-25): ";
        cin >> K;
        cin.ignore(); // Clear newline
        CaesarCipher cipher(K);
        string decrypted = cipher.decrypt(ciphertext);
        cout << "Plaintext: " << decrypted << endl;
    }
    else if (choice == 3)
    {
        // Brute Force Attack
        string ciphertext;
        cout << "Enter ciphertext: ";
        getline(cin, ciphertext);
        CaesarCipher cipher(0); // Dummy cipher, not used for brute force
        vector<pair<int, string>> results = cipher.bruteForceAttack(ciphertext);
        cout << "All possible plaintexts (identify the correct one manually):" <<
endl;

        for (const auto &result : results)
        {
            cout << "Key " << result.first << ": " << result.second << endl;
        }
    }
}
```

```
    }  
    }  
    else if (choice == 4)  
    {  
        cout << "Exiting program." << endl;  
    }  
    else  
    {  
        cout << "Invalid choice. Please select 1-4." << endl;  
    }  
} while (choice != 4);  
  
return 0;  
}
```

Code Explanation:

- 1. Class-Based Structure:** The code encapsulates functionality within the `CaesarCipher` class, promoting an object-oriented design by keeping the `shift` variable private and accessible only through public methods.
- 2. Constructor & Normalization:** The constructor `CaesarCipher(int s)` initializes the cipher and uses `s % 26` to ensure that shift values larger than 26 remain within the valid alphabet range.
- 3. Character Math:** The `encrypt` method utilizes ASCII arithmetic. By subtracting the base character ('A' or 'a') from the input character, it converts the letter to a 0-25 index.
- 4. Encryption Formula:** The method implements the mathematical definition of the cipher: $E(x) = (x + k) \bmod 26$, allowing the alphabet to wrap (e.g., 'Z' shifted by 1 becomes 'A').
- 5. Decryption Logic:** The `decrypt` method reverses the process with the formula $D(x) = (x - k + 26) \bmod 26$. The inclusion of 26 ensures that results are positive prior to applying the modulo operation, thus avoiding negative indexing errors in C++.
- 6. Case Preservation:** The code checks `isupper(c)` to determine the appropriate ASCII base, ensuring that uppercase and lowercase letters maintain their case after shifting.
- 7. Input Handling:** The main function employs `getline(cin, ...)` instead of `cin >> ...`, enabling users to input sentences with spaces (e.g., "Hello World") without interruption at the first whitespace.
- 8. Brute Force Mechanism:** The `bruteForceAttack` method illustrates a "ciphertext-only attack," iterating through all possible keys (1 to 25), decrypting the text with each key, and storing the results in a vector for analysis.
- 9. Non-Alphabetic Character Handling:** The `isalpha` check ensures that non-alphabetic characters, including spaces, numbers, and punctuation marks, remain unchanged in the result string, preserving the structure of the input.
- 10. Menu-Driven Interface:** The main function utilizes a do-while loop and a switch-case structure (or if/else if statements) to facilitate multiple operations without restarting the program.

2. Output

Encryption

```
Menu:
1. Encrypt
2. Decrypt
3. Brute Force Attack
4. Exit
Choose an option (1-4): 1
Enter plaintext: Usama Arshad
Enter key K (1-25): 8
Ciphertext: Caiui Izapil
```

Decryption

In decryption, the key was intentionally set incorrectly to check the validity of the algorithm.

```
Menu:
1. Encrypt
2. Decrypt
3. Brute Force Attack
4. Exit
Choose an option (1-4): 2
Enter ciphertext: Caiui Izapil
Enter key K (1-25): 7
Plaintext: Vtbnb Bstibe
```

Decryption using Brute Force

In a brute force approach, we can see that the correct text was found at the position where the Key was 8.

```
Menu:
1. Encrypt
2. Decrypt
3. Brute Force Attack
4. Exit
Choose an option (1-4): 3
Enter ciphertext: Caiui Izapil
All possible plaintexts (identify the correct one manually):
Key 1: Bzhth Hyzohk
Key 2: AygsG Gxyngj
Key 3: Zxfrf Fwxmfi
Key 4: Yweqe Ewileh
Key 5: Xvdpd DuvkdG
Key 6: Wucoc Ctujcf
Key 7: Vtbnb Bstibe
Key 8: Usama Arshad
Key 9: TrzLz Zqrgzc
Key 10: Sqyky Ypqfyb
Key 11: Rpxjx Xopexa
Key 12: QowiW Wnodwz
Key 13: Pnvhv Vmncvy
Key 14: Omugu Ulmbux
Key 15: Nltft Tklatw
Key 16: MkSES Sjkzsv
Key 17: Ljrdr Rijyru
Key 18: Kiqcq Qhixqt
Key 19: Jhpbp Pghwps
Key 20: Igoao Ofgvor
Key 21: Hfnzn Nefunq
Key 22: Gemym Mdetmp
Key 23: Fdlxl Lcdslo
Key 24: Eckwk KbcrkN
Key 25: Dbjvj Jabqjm
```

3. Observation

The code effectively manages input/output operations, ignoring punctuation and numbers while shifting only alphabetic characters. The menu loop continues until the user opts to exit.

4. Result

Upon selecting option one, plaintext is converted into ciphertext using the specified key. Option three produces twenty-five lines of output, with one line potentially being coherent English.

5. Applications

- **Cryptography Education:** Useful for teaching modular arithmetic concepts.
- **ROT13:** A specific variation of the cipher (key 13) commonly used on online forums to conceal spoilers.
- **Puzzles:** Can be employed in escape rooms or simple data obfuscation scenarios where high security is not a priority.

6. Advantages

- **Ease of Implementation:** The logic requires minimal computing resources.
- **Reversibility:** Decryption is straightforward if the key is known.
- **Transparency:** Clearly demonstrates substitution cipher mechanics for educational purposes.

7. Disadvantages

- **Ease of Implementation:** The logic requires minimal computing resources.
- **Reversibility:** Decryption is straightforward if the key is known.
- **Transparency:** Clearly demonstrates substitution cipher mechanics for educational purposes.

8. Conclusion

This C++ implementation provides a framework for understanding symmetric encryption. While the Caesar Cipher may not be suitable for modern security needs, it serves as a tool for teaching string manipulation and algorithm concepts.

THE END
